
OCA – PW AI Integration Proposal

A Framework for Unified AI-Assisted
Development on NOAA RDHPCS

Terry McGuinness
OMD Code Assist Team
NOAA / NWS / NCEP / EMC
<https://github.com/NOAA-EMC/global-workflow>
August 28, 2026 • Document Version 1.0

Abstract

The Operations and Maintenance Division (OMD) Code Assist system (OCA) provides Model Context Protocol (MCP) based AI tooling for NOAA’s Global Forecast System (GFS) development on the Global Workflow codebase. This proposal presents an architecture for integrating OCA’s MCP-RAG server with Parallel Works (PW) ACTIVATE’s AI gateway, `pw code` terminal agent, and self-hosted inference via vLLM. The result is a governed, budget-tracked, air-gap-compatible AI development platform that operates entirely within NOAA’s RDHPCS environment while leveraging commercial LLM providers (Anthropic, OpenAI) or locally served open models through a single unified gateway.

Keywords: MCP, RAG, vLLM, NOAA, HPC, Global Workflow, Parallel Works, AI Gateway, Code Assist

Note. This document is the LaTeX-rendered PDF of `oca_pw_ai_integration_proposal.md`, the Markdown source shipped alongside for portability on HPC head nodes where TeX toolchains are not available. Section numbering, citation conventions, and figure references follow academic paper formatting.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Scope	2

2	Background: The OCA MCP-RAG System	2
2.1	System Overview	2
2.2	Architecture	3
2.3	Tool Inventory	3
2.4	RAG Knowledge Base	4
2.5	Development History	5
3	Parallel Works ACTIVATE AI Platform	5
3.1	Platform Overview	5
3.2	<code>pw code</code> : Terminal AI Coding Agent	5
3.3	AI Gateway Architecture	6
3.4	Tool-Calling Emulation	6
3.5	Connectivity: Sessions, Tunnels, and Endpoints	7
4	Integration Architecture	7
4.1	Target Architecture	7
4.2	MCP Configuration for <code>pw code</code>	8
4.3	Custom Agent Definition	8
5	Connectivity Patterns for RDHPCS	8
5.1	Current State (Working)	9
5.2	Investigated but Non-Functional	9
5.3	Proposed: <code>pw forward</code> for Auto-Reconnection	9
6	LLM Provider Options and Billing	9
6.1	Provider Comparison	10
6.2	PW Budget Allocation Flow	10
6.3	GitHub Copilot BYOK Integration	11
6.4	Self-Hosted vLLM on RDHPCS	11
7	Implementation Roadmap	11
8	Existing Infrastructure Inventory	12
8.1	Processes on Gaea C6 (gaea62)	13
8.2	MCP Gateway (AWS Rocky 9)	13
8.3	PW CLI	13
9	Risk Assessment	14
10	Conclusion	14
	References	14

1 Introduction

1.1 Motivation

NOAA’s Global Workflow is a production weather forecasting framework supporting the Global Forecast System (GFS), Global Ensemble Forecast System (GEFS), Seasonal Forecast System (SFS), and Global Climate Analysis Forecast System (GCAFS). The codebase spans approximately 89 job scripts, 83 execution scripts, 78 utility modules, and extensive Fortran/C/Python source across dozens of submodules. Development occurs on federally managed RDHPCS platforms (Gaea C6, Hera, Hercules, WCOS2) where internet access is restricted, software installation requires coordination, and GPU resources are shared.

AI-assisted development tools—GitHub Copilot, Claude Code, Cursor—have demonstrated significant productivity gains in open-source and commercial software engineering. However, deploying these tools on RDHPCS systems presents three fundamental challenges:

1. **Network isolation.** HPC login and compute nodes cannot reach commercial API endpoints directly. Tunneler connections are required.
2. **Governance.** Federal systems require auditable access controls, usage tracking, and budget management for any cloud-consumed service.
3. **Domain specificity.** General-purpose LLMs lack knowledge of NOAA’s operational conventions, Rocoto workflow patterns, wxflow integration points, UFS model architecture, and EE2 compliance requirements.

The OCA MCP-RAG system addresses challenge (3). This proposal extends that foundation to address challenges (1) and (2) by integrating with Parallel Works’ ACTIVATE AI platform.

1.2 Scope

This document covers:

- Background on the OCA MCP-RAG system (Section 2)
- Parallel Works ACTIVATE AI platform capabilities (Section 3)
- Integration architecture: MCP + PW AI gateway + vLLM (Section 4)
- Connectivity patterns for RDHPCS environments (Section 5)
- LLM provider options and billing models (Section 6)
- Implementation roadmap (Section 7)

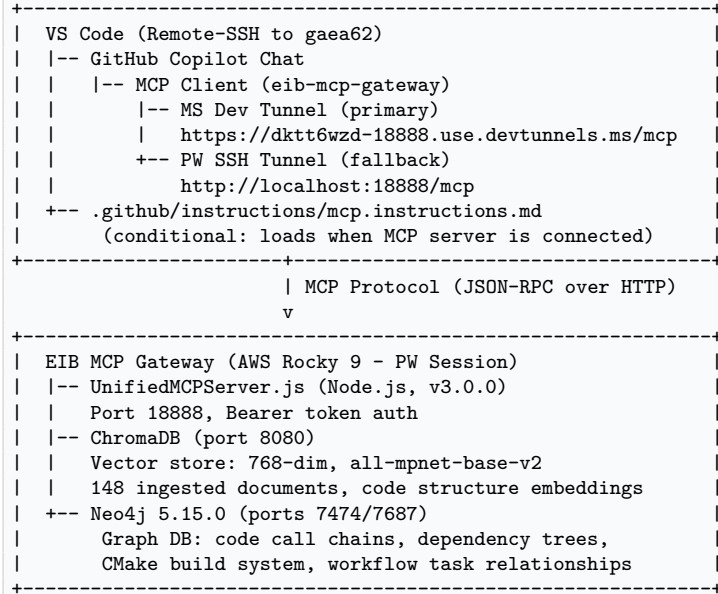
2 Background: The OCA MCP-RAG System

2.1 System Overview

The OMD Code Assist (OCA) system is a Model Context Protocol server with Retrieval-Augmented Generation (RAG) capabilities, purpose-built for the Global Workflow codebase. The system has been under active development since July 2024, progressing through multiple phases from initial RAG prototyping to a production-grade MCP server (v3.0.0) with 39 tools across 8 functional modules.

The MCP server is designated `eib-mcp-gateway` and runs on a Parallel Works managed AWS Rocky 9 compute session (`emcmcpawsrocky9copyfour`), accessible to VS Code on RDHPCS systems via tunneled HTTP connections.

2.2 Architecture



2.3 Tool Inventory

The MCP server exposes 39 tools organized into 8 modules (Table 1).

Table 1: OCA MCP tool inventory across eight functional modules.

Module	Tools	Purpose
Workflow Information (6)	get_server_info, get_workflow_structure, list_job_scripts, get_job_details, explain_workflow_component, get_system_configs	Navigate Global Workflow structure
Code Analysis (7)	get_code_context, find_callers_callees, find_dependencies, find_similar_code, extract_code_for_analysis, analyze_code_structure, find_related_files	Deep code understanding
Semantic Search (4)	search_documentation, search_architecture, search_ee2_standards, find_env_dependencies	RAG-powered knowledge retrieval
EE2 Compliance (5)	analyze_ee2_compliance, scan_repository_compliance, validate_sdd_compliance, generate_compliance_report, search_ee2_standards	NOAA standards checking
Operational (6)	trace_execution_path, trace_data_flow, trace_full_execution_chain, analyze_workflow_dependencies, get_operational_guidance, explain_with_context	Production system analysis
GraphRAG (5)	describe_component, get_quality_metrics, get_health_trend, get_change_impact, check_knowledge_integrity	Graph-powered analysis
GitHub Integration (3)	search_issues, get_pull_requests, extract_ci_error_signal	Repository operations
Utility (3)	mcp_health_check, list_ingested_urls, get_ingested_urls_array	System management

2.4 RAG Knowledge Base

The knowledge base is organized in three tiers:

- **Tier 1 (Authoritative):** Production source code, official NOAA documentation, EE2/SDLC standards.
- **Tier 2 (Supplementary):** Architecture decisions, design documents, historical context.
- **Tier 3 (Reference):** External UFS documentation, community resources.

The Phase 2 ingestion pipeline processed 148 documents covering UFS documentation, RT/CI framework guides, Rocoto workflow management, wxflow library internals, GDAS/GSI data assimilation, wave model integration, aerosol modeling, and build system architecture.

2.5 Development History

Table 2: OCA MCP-RAG development milestones.

Date	Milestone
Jul 2024	Initial RAG server prototype
Aug 2024	GitHub MCP integration
Sep 2024	Neo4j graph database for code structure
Oct 2024	Python/Shell code ingestion pipeline
Nov 2024	CMake build system ingestion
Dec 2024	Embedding model upgrade (all-mpnet-base-v2)
Jan 2025	Phase 2: Unified documentation ingestion (148 docs)
Mar 2025	EE2 compliance reporting tools
Jun 2025	MCP instructions file merged to <code>develop</code> branch
Aug 2025	Unified MCP Server v3.0.0
2026	PW AI gateway integration exploration (this proposal)

3 Parallel Works ACTIVATE AI Platform

3.1 Platform Overview

Parallel Works ACTIVATE is a hybrid, multi-cloud HPC platform deployed at NOAA for managing compute resources, sessions, and workflows across RDHPCS systems. The PW CLI (`pw`, v7.94.0, Go binary from github.com/parallelworks/core) is installed on Gaea C6 at `/ncrc/home2/Terry.McGuinness/pw/p`

ACTIVATE’s AI subsystem, introduced as a governed AI gateway, provides:

- Centralized model access via an OpenAI-compatible API endpoint
- Token-level usage metering against named budget allocations
- Multi-provider support (Azure OpenAI, Anthropic, OpenAI, Grok, Groq, Together, Fireworks, Mistral, DeepSeek, custom OpenAI-compatible, AWS Bedrock, Azure AI Foundry)
- Session tunnel providers for air-gapped model serving
- Tool-calling emulation for endpoints without native tool support

3.2 `pw code`: Terminal AI Coding Agent

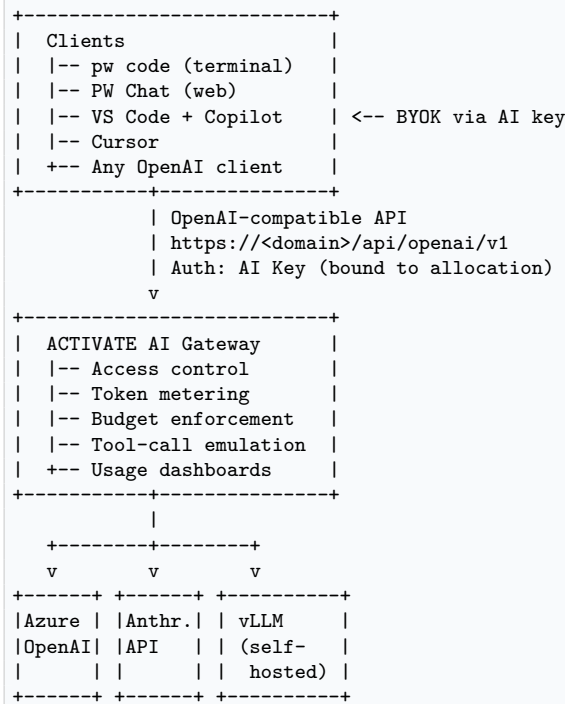
`pw code` is Parallel Works’ built-in AI coding agent—a terminal-based tool comparable to Claude Code or OpenAI Codex CLI. Key capabilities are summarized in Table 3.

Table 3: Key capabilities of the `pw code` terminal agent.

Feature	Description
Full-screen TUI	Bubble Tea framework with markdown rendering, syntax highlighting, diff views
Tool-calling agent	ReadFile, WriteFile, EditFile, Bash, GrepSearch, GlobSearch, WebFetch, Task
MCP integration	Native MCP client via <code>pw code mcp add</code> (same config format as Claude Code)
Daemon architecture	Background daemon hosts sessions; TUI detaches/reattaches
Remote attach	<code>pw code attach <resource></code> steers agents on remote HPC nodes
Session persistence	Saved under <code>~/.local/state/pw/code-sessions</code> , resumable
Subagents	Parallel child agents for concurrent work
Custom agents	<code>.agents/</code> directory with YAML frontmatter + markdown specialization
Skills / Hooks	Reusable procedures and lifecycle event handlers
Plan mode	Review-before-execute workflow

3.3 AI Gateway Architecture

Every request—whether from PW Chat, `pw code`, or an external tool authenticated with an AI key—is proxied through the ACTIVATE AI gateway:



3.4 Tool-Calling Emulation

For self-hosted models (vLLM, Ollama, TGI) that lack native tool-calling APIs, the ACTIVATE gateway provides transparent emulation:

1. Tool schemas are injected into the prompt with a per-request nonce.
2. The model responds with a JSON envelope (`protocol: parallelworks.tool-call.v1`).
3. The gateway validates framing values, repairs common format variations, and converts to standard OpenAI `tool_calls`.
4. Near-miss envelopes get one corrective retry before failing.

5. Tool results are fenced as untrusted data with nonce markers.

This means `pw code` can use all its built-in tools **and** connected MCP server tools even when backed by an open-weights model served by vLLM.

3.5 Connectivity: Sessions, Tunnels, and Endpoints

PW provides multiple connectivity mechanisms relevant to MCP integration (Table 4).

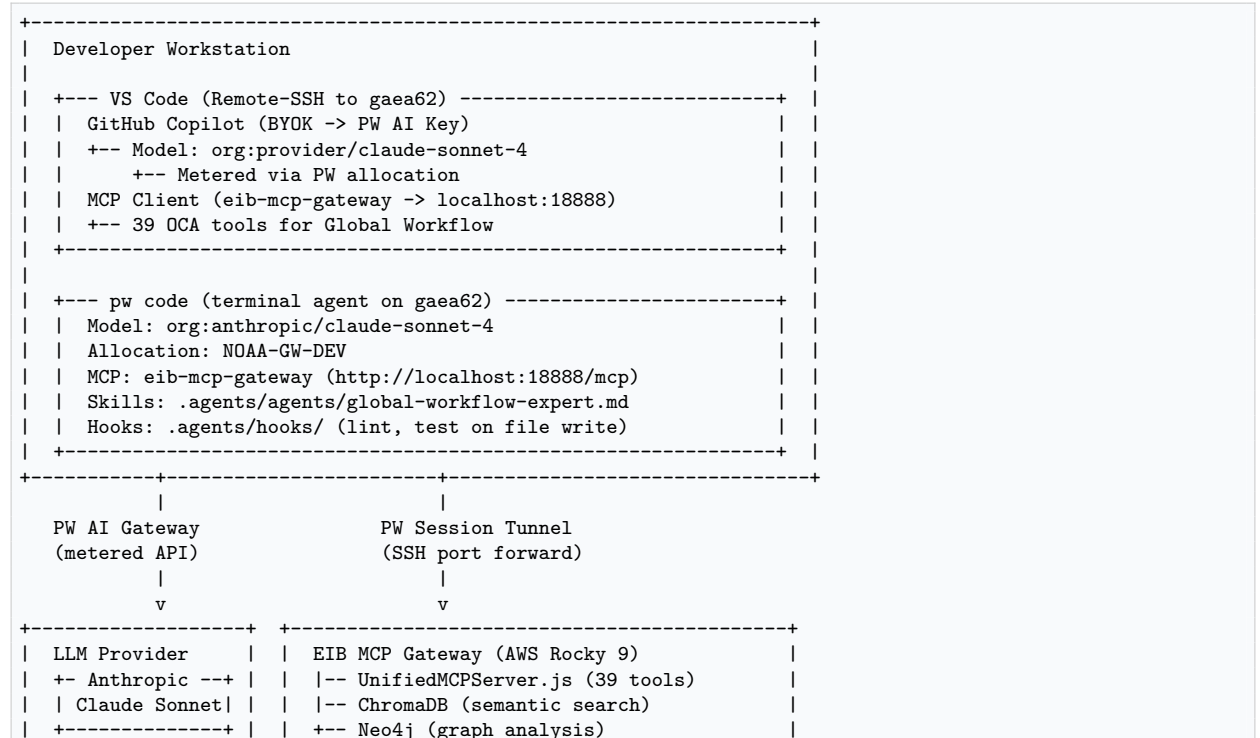
Table 4: PW connectivity mechanisms relevant to MCP integration.

Mechanism	Command	Use Case
Session connect	<code>pw sessions connect <name></code>	SSH tunnel to a PW session (current MCP path)
Port forward	<code>pw forward -L <p>:localhost:<p> <res></code>	Auto-reconnecting SSH tunnel
Endpoint (HTTP)	<code>pw endpoints http --openai <port></code>	Expose local server as platform AI model
VS Code remote	<code>pw vscode <resource> [path]</code>	Open VS Code via Remote-SSH
Code attach	<code>pw code attach <resource></code>	Steer terminal agent on remote node
SOCKS proxy	<code>pw forward -D <port> <resource></code>	Full network proxy via remote

4 Integration Architecture

4.1 Target Architecture

The proposed architecture unifies OCA’s domain-specific MCP tools with PW’s governed AI gateway and optional self-hosted inference:



```
| +- vLLM -----+ | +-----+
| | Llama 3 70B | |
| | (GPU node) | |
| +-----+ |
+-----+
```

4.2 MCP Configuration for pw code

The `pw code` agent reads MCP server configuration from `.mcp.json` in the workspace root, using the same format as Claude Code and VS Code:

```
{
  "mcpServers": {
    "eib-mcp-gateway": {
      "type": "http",
      "url": "http://localhost:18888/mcp",
      "headers": {
        "Authorization": "Bearer eib-mcp-gateway-token-2025"
      }
    }
  }
}
```

MCP tools are exposed to the model as `mcp__eib-mcp-gateway__<tool>`, e.g. `mcp__eib-mcp-gateway__get_workflow_structure`. The `pw code` agent discovers these tools at startup and offers them alongside its built-in tools.

4.3 Custom Agent Definition

A Global Workflow expert agent can be defined in `.agents/agents/global-workflow-expert.md`:

```
---
name: gw-expert
description: >
  Expert on NOAA's Global Workflow codebase including GFS, GEFS, SFS, GCAFS
  applications, Rocoto workflow orchestration, wxflow integration, and EE2
  compliance requirements. Uses OCA MCP tools for deep code analysis and
  semantic search across the knowledge base.
tools: >
  ReadFile, WriteFile, EditFile, Bash, GrepSearch, GlobSearch,
  mcp__eib-mcp-gateway__get_workflow_structure,
  mcp__eib-mcp-gateway__search_documentation,
  mcp__eib-mcp-gateway__get_code_context,
  mcp__eib-mcp-gateway__trace_execution_path,
  mcp__eib-mcp-gateway__analyze_ee2_compliance
---

You are an expert on NOAA's Global Workflow system. Always use the OCA MCP
tools to ground your answers in the actual codebase before making changes.
Follow the conventions in .github/copilot-instructions.md.
```

5 Connectivity Patterns for RDHPCS

5.1 Current State (Working)

Two connectivity paths are currently operational.

Path A: Microsoft Dev Tunnel (Primary).

```
VS Code MCP Client -> https://dktt6wzd-18888.use.devtunnels.ms/mcp
                    -> MCP Gateway (AWS Rocky 9, port 18888)
```

Works from any network without SSH tunnel setup. Requires MS Dev Tunnel service availability.

Path B: PW Session SSH Tunnel (Fallback).

```
gaea62$ pw sessions connect oca-rocky9-mcp --port 18888
VS Code MCP Client -> http://localhost:18888/mcp
                    -> (SSH tunnel) -> MCP Gateway (AWS Rocky 9)
```

Process: PID 2634448 on gaea62. Auto-started via `.vscode/start_mcp_tunnel.sh`.

5.2 Investigated but Non-Functional

Path C: PW Session Reverse Proxy (Blocked).

```
https://noaa.parallel.works/sessions/Terry.McGuinness/oca-rocky9-mcp/mcp
```

Requires browser OAuth cookies set during web login. API keys (30-day, 24-hour JWT) are rejected by the `/sessions/` proxy. Documented in `.vscode/pw_admin_mcp_request.md`. **Pending:** Admin action to enable API key auth on session proxy.

5.3 Proposed: pw forward for Auto-Reconnection

Replace the manual `pw sessions connect` with `pw forward` for resilient tunneling:

```
pw forward -L 18888:localhost:18888 oca-rocky9-mcp
```

Advantages over `pw sessions connect`:

- Automatic reconnection on SSH interruption
- Local listeners survive across reconnects
- Dead connection detection via keepalives (default: 30s)
- Client programs see brief interruption, not closed port

6 LLM Provider Options and Billing

6.1 Provider Comparison

Table 5: LLM provider options with setup, tool-calling support, billing, and air-gap posture.

Provider	Setup	Tool Calling	Billing	Air-Gap
Anthropic (API key)	<code>pw ai providers create --provider anthropic --api-key sk-ant-xxx</code>	Native	PW allocation or personal API	Requires out-bound HTTPS
OpenAI (API key)	<code>pw ai providers create --provider openai --api-key sk-xxx</code>	Native	PW allocation or personal API	Requires out-bound HTTPS
Codex subscription	<code>pw ai codex connect --name personal</code>	Native	Personal ChatGPT subscription quota	Requires out-bound HTTPS
Azure OpenAI	Admin provisions via <code>--provider azure-foundry</code>	Native	PW allocation (org billing group)	Azure private endpoint possible
AWS Bedrock	<code>--provider bedrock --cloud-account <acct></code>	Native	PW allocation (cloud account)	VPC endpoint possible
vLLM (self-hosted)	<code>pw endpoints http --openai <port></code>	Emulated (gateway)	PW allocation (compute cost only)	Fully air-gapped
Ollama (self-hosted)	<code>pw endpoints http --openai --rewrite-host 11434</code>	Emulated (gateway)	PW allocation (compute cost only)	Fully air-gapped

6.2 PW Budget Allocation Flow

```

Org Admin
|
|-- Creates AI Provider (Anthropic, vLLM, etc.)
|-- Creates Budget Allocation ("NOAA-GW-DEV", $X limit)
|-- Assigns allocation to user/group
+-- Shares provider via aiprovider:use permission
    |
    v
Developer
|
|-- pw code --allocation NOAA-GW-DEV org:provider/model
|-- pw ai models ls      (lists available models)
|-- /allocation          (picks allocation interactively)
+-- /usage               (shows current session spend)
    |
    v
AI Usage Dashboard
|-- Total spend per period (7/30/90 days)
|-- Tokens used (input/output breakdown)
|-- Breakdown by model, user, allocation, API key
+-- Budget enforcement (rejects when exhausted)

```

6.3 GitHub Copilot BYOK Integration

GitHub Copilot’s Bring Your Own Key (BYOK) feature can route through the PW gateway for metered billing:

```
GitHub Copilot (VS Code)
-> PW AI Key (bound to allocation)
-> https://noaa.parallel.works/api/openai/v1
-> ACTIVATE AI Gateway (metered)
-> Backend Provider (Anthropic, Azure, vLLM)
```

Configuration in VS Code:

```
{
  "github.copilot.advanced": {
    "baseUrl": "https://noaa.parallel.works/api/openai/v1"
  }
}
```

This provides unified billing across all AI tools—Copilot completions, Copilot Chat, `pw code` agent sessions, and any other OpenAI-compatible client—under a single PW allocation with full audit trail.

6.4 Self-Hosted vLLM on RDHPCS

For fully air-gapped operation or when commercial API access is restricted:

```
# On a GPU compute node (Slurm job or PW session)
vllm serve meta-llama/Llama-3-70B-Instruct \
  --api-key my-secret \
  --port 8000 \
  --tensor-parallel-size 4

# Expose as PW AI provider (auto-registers in gateway)
pw endpoints http --openai --name gaea-vllm \
  --auth-file /path/to/api-key 8000
```

The gateway’s tool-calling emulation layer enables `pw code` to use all built-in and MCP tools with the self-hosted model, despite vLLM not implementing the OpenAI tools API natively. The emulation protocol (`parallelworks.tool-call.v1`) uses per-request nonces and toolset digests to prevent prompt injection from tool output.

7 Implementation Roadmap

Phase 1: Immediate (Week 1–2)

Task	Action	Prerequisite
Create Anthropic provider	<code>pw ai providers create --provider anthropic --api-key <key></code>	Anthropic API key
Validate <code>pw</code> code with MCP	Create <code>.mcp.json</code> , run <code>pw</code> code in workspace	Provider created
Upgrade tunnel to <code>pw forward</code>	Replace <code>pw sessions connect</code> with <code>pw forward -L 18888:...</code>	None
Create custom agent definition	<code>.agents/agents/global-workflow-expert.md</code>	None

Phase 2: Org-Governed (Week 3–4)

Task	Action	Prerequisite
Request PW admin provider setup	Admin creates org-shared Anthropic or Azure OpenAI provider	Org admin access
Budget allocation	Admin creates allocation (e.g., NOAA-GW-DEV) and assigns to team	Provider exists
AI key for Copilot BYOK	Create key at AI > AI Keys, bind to allocation	Allocation exists
Configure Copilot BYOK	Point Copilot at PW gateway base URL with AI key	AI key created

Phase 3: Self-Hosted LLM (Month 2)

Task	Action	Prerequisite
GPU allocation	Request GPU node access on Gaea C6 or cloud cluster	HPC account
Deploy vLLM	Slurm job or PW session with vLLM serving Llama 3 70B	GPU access
Register as provider	<code>pw endpoints http --openai 8000</code>	vLLM running
Validate emulated tools	Test <code>pw</code> code tool calling with self-hosted model	Endpoint registered

Phase 4: Platform Maturity (Month 3+)

Task	Action
PW session proxy auth	Admin enables API key authentication on <code>/sessions/</code> proxy
Skills library	Create reusable OCA skills in <code>.agents/agents/skills/</code>
CI integration	<code>pw code -p</code> one-shot runs in CI for automated code review
Usage analytics	Review AI Usage dashboard, optimize model / allocation strategy

8 Existing Infrastructure Inventory

8.1 Processes on Gaea C6 (gaea62)

PID	Process	Purpose
2634448	<code>pw sessions connect oca-rocky9-mcp --port 18888</code>	SSH tunnel to MCP gateway
1939122	<code>python3.12 sdp-mcp-server/server.py --port 8765</code>	GFDL ServiceDesk Plus MCP (Seth Underwood, unrelated)

8.2 MCP Gateway (AWS Rocky 9)

- **Server:** UnifiedMCPServer.js v3.0.0 (Node.js)
- **Port:** 18888
- **Auth:** Bearer token (eib-mcp-gateway-token-2025)
- **ChromaDB:** Port 8080, 768-dim embeddings
- **Neo4j:** Ports 7474 (HTTP) / 7687 (Bolt)
- **Session:** oca-rocky9-mcp on emcmcpawsrocky9copyfour

8.3 PW CLI

- **Version:** v7.94.0
- **Binary:** /ncrc/home2/Terry.McGuinness/pw/pw
- **Source:** github.com/parallelworks/core (private)
- **SDK:** github.com/parallelworks/sdk/go/v7
- **Platform:** noaa.parallel.works

9 Risk Assessment

Table 6: Risk register with likelihood, impact, and mitigation.

Risk	Likelihood	Impact	Mitigation
PW admin delays provider/allocation setup	Medium	Blocks Phase 2	Phase 1 uses personal provider; document admin request clearly
Network instability on SSH tunnels	Medium	MCP tool failures	<code>pw forward</code> auto-reconnect; MS Dev Tunnel as fallback
vLLM tool-calling emulation accuracy	Medium	Degraded agent performance	Use commercial models (Anthropic) for complex tasks; vLLM for routine
Budget exhaustion mid-sprint	Low	Work stoppage	Monitor <code>/usage</code> ; set allocation alerts; right-size model selection
MCP gateway session stops	Low	All MCP tools unavailable	<code>--keep</code> flag on endpoint; health monitoring via <code>mcp_health_check</code> tool
RDHPCS GPU availability	Medium	Blocks self-hosted LLM	Cloud fallback via commercial providers

10 Conclusion

The OCA MCP-RAG system and Parallel Works ACTIVATE AI gateway are complementary technologies that, when integrated, provide a complete AI-assisted development platform for NOAA’s Global Workflow. OCA contributes domain-specific knowledge retrieval and code analysis tools via MCP. PW ACTIVATE contributes governed LLM access with budget tracking, multi-provider support, and native MCP client integration through `pw code`.

The immediate path—creating a personal Anthropic provider and connecting `pw code` to the existing MCP gateway—requires no admin intervention and can be operational within hours. The governed path—org-shared providers with budget allocations feeding both `pw code` and GitHub Copilot BYOK—requires admin action but delivers unified billing and audit trails. The self-hosted path—vLLM on RDHPCS GPU nodes with emulated tool calling—provides full air-gap capability for sensitive operations.

All three paths share the same MCP tools, the same workspace configuration, and the same developer experience.

References

1. Parallel Works ACTIVATE Documentation. <https://parallelworks.com/docs>
2. Parallel Works AI Providers. <https://parallelworks.com/docs/ai/ai-providers>
3. Parallel Works `pw code` Documentation. <https://parallelworks.com/docs/ai/code>
4. Parallel Works Session Tunnels. <https://parallelworks.com/docs/ai/ai-providers/session-tunnels>
5. Parallel Works Tool Calling & Emulation. <https://parallelworks.com/docs/ai/ai-providers/tool-calling>

6. Parallel Works Connect External Tools. <https://parallelworks.com/docs/ai/ai-providers/connect>
7. Parallel Works LLM Knowledge Corpus. <https://parallelworks.com/llms-full.txt>
8. Model Context Protocol Specification. <https://modelcontextprotocol.io>
9. NOAA Global Workflow Repository. <https://github.com/NOAA-EMC/global-workflow>
10. vLLM: Easy, Fast, and Cheap LLM Serving. <https://docs.vllm.ai>

Document Version: 1.0 • **Date:** August 28, 2026 • **Author:** Terry McGuinness / OMD Code Assist Team • **Classification:** For Internal Distribution